Transcript

Week 18: Convolution Neural Networks

Video 1 - Convolution Neural Networks

How do machines recognize faces, read traffic signs, or detect objects in a photo—almost instantly?

Convolutional Neural Networks or CNNs are behind these breakthroughs in computer vision. Built on deep learning foundations, they process grid-like data—like images—through specialized operations such as convolution, pooling, and feature extraction. CNNs enable systems to identify patterns, classify visuals, and segment scenes with exceptional accuracy by mimicking how humans perceive visuals.

Convolutional Neural Networks were developed to overcome the limitations of traditional neural networks in processing image data. Unlike standard networks, CNNs use operations like convolution, pooling, and padding to capture spatial patterns. They apply filters and strides across image grids, gradually building a hierarchy of visual features. Popular architectures such as AlexNet, VGGNet, and ResNet demonstrate how CNNs have evolved to solve increasingly complex visual tasks with speed and accuracy.

Let's decode how CNNs see the world—layer by layer.

Video 2 - CNN Basics

Machines can do more than just follow rules—they can discover patterns on their own. That's the power of unsupervised learning. Before we explore how Convolutional Neural Networks learn from images, let's understand how data can speak for itself, even without labels. This sets the foundation for deeper learning with visual inputs.

Before we dive into Convolutional Neural Networks, let's briefly look at Deep Neural Networks, or D–N–N's. These networks use many layers to learn patterns in complex data. But when we use D–N–N's with images, we have to flatten the image into a one-dimensional vector. This loses important spatial information—like how pixels relate to each other in the image. D–N–N's also struggle to capture local features, and they become inefficient with very large inputs. That's where C–N–N's come in. They are built to handle image data and solve these specific challenges more effectively.

To understand why standard neural networks aren't ideal for image tasks, we need to look at how they handle data. Images are high-dimensional, meaning each pixel carries unique information. To fit into dense layers, we flatten the image into a one-dimensional vector. But this flattens out the structure too, removing spatial relationships—like how one pixel relates to its neighbour. Every pixel must connect to every neuron in the next layer, which increases the number of parameters and slows down learning. Local features like edges and textures get lost in the process. These challenges led to the development of networks better suited for

image data—like Convolutional Neural Networks.

Convolutional Neural Networks are built to handle image data with precision and efficiency. Unlike traditional neural networks that flatten images and lose spatial structure, CNNs use convolutional layers with small filters that slide across the image to detect patterns like edges, textures, and shapes. This method captures the spatial relationships between pixels, helping CNNs retain crucial visual details. By learning features in a structured way, CNNs reduce the number of parameters while improving accuracy, making them ideal for tasks like image recognition and classification.

Dense layers connect every neuron to all others in the next layer, which can work well for simple tasks. But when it comes to image data, this approach creates a large number of parameters and ignores spatial structure. Convolutional layers solve this by focusing on small, local regions of the image using sliding filters. This localized approach captures patterns like edges while keeping spatial relationships intact—making the model more efficient and scalable for visual tasks.

Now that you know the basics of C–N–Ns, let's walk through how they classify images.
In **Step one**, convolutional layers scan the image with filters to detect features like edges and textures.
In **Step two**, pooling layers reduce the size of the data while keeping the most important features.
Finally, in **Step three**, the filtered and pooled data is passed into fully connected layers, which combine the information and predict the correct class—like "cat" or "dog."

Convolutional Neural Networks offer clear advantages over traditional neural networks, especially when working with image data. Because CNNs connect only to small regions of the image, they reduce the number of parameters while still capturing useful patterns like edges and textures. This makes them efficient, even with large and complex image data. CNNs also preserve the spatial layout of features, helping models understand where patterns occur. That's why CNNs are used in real-world tasks like image recognition, object detection, and even in medical imaging where accuracy is critical.

Think about where you've seen image recognition in action—now imagine building it yourself. With CNNs, you have the tools to turn complex images into smart, accurate predictions. Ready to apply this in the real world?

Video 3 - Core Operations in CNNs

Behind every smart image recognition system lies a powerful design. CNNs are not just layers stacked together—they are carefully crafted to spot patterns, preserve details, and make sense of complex visuals. Let's uncover the architecture that transforms raw pixels into real-world predictions.

Convolutional Neural Networks are built to handle image data in its original two-dimensional form.
Instead of flattening the image into a single line of values, C–N–Ns preserve the 2D structure. This helps retain spatial relationships between pixels, which is important for recognising

patterns.
Each pixel in the image holds a value between **0 and 255** that represents the brightness or colour at that point.
C–N–Ns use filters—also called matrices—that move across the grid to find patterns like edges or textures.
These filters help the network focus on important features in the image.

In a C-N-N, filters—also called matrices or kernels—help detect patterns in an image. A filter is a small grid, like three-by-three or five-by-five, that slides across the image. Each number in the filter acts as a weight and multiplies the pixel values underneath it. The results are added up to form a single output, and this builds what we call a **feature map**. This feature map highlights key parts of the image, like edges, textures, or corners, that the C-N-N will learn from.

Filters in a C–N–N are designed for specific tasks, such as detecting edges, blurring details, or sharpening important features in an image.
Let's look at a few common types.

An **edge detection** filter—for example, using values like minus one, minus one, minus one, zero, zero, zero, one, one, one—helps highlight sharp changes in brightness. This allows the model to detect edges and outlines.

Next, a **blur** filter uses small values—such as one over nine repeated across all entries—to soften the image. By averaging nearby pixels, it smooths out noise and reduces harsh transitions.

In contrast, a **sharpen** filter might include values like zero, minus one, zero; minus one, five, minus one; zero, minus one, zero. These numbers boost key differences in the image, making important features stand out more clearly.

Together, these filters allow CNNs to focus on different aspects of the image—helping the model detect, smooth, or highlight patterns and better understand what it's seeing.

In convolutional neural networks, a filter slides over the image to perform a mathematical operation called convolution. At each step, the filter multiplies its values with the underlying image pixels and sums the result, producing a single value that represents that specific region. This process creates a new matrix called the feature map, which highlights the patterns captured by the filter—like edges or textures. Convolution reduces the image's dimensions while preserving essential visual features, making the data easier for the network to process.

Let's take a closer look at how convolution works in practice. Imagine a three-by-three filter scanning over a five-by-five image. At each step, it multiplies the filter's numbers with the underlying pixel values, adds the results, and records a single value in the feature map. This process repeats across the image, capturing key patterns like edges or textures. The result is a smaller feature map—a simplified version of the image that preserves what the filter is designed to detect. This makes it easier for C-N-N's to understand shapes, objects, and patterns with reduced complexity.

Pooling is an important step that follows convolution in Convolutional Neural Networks, or C-N-N's.

After convolution, pooling reduces the size of feature maps, creating a simpler version of the data.
By shrinking the spatial dimensions, pooling helps the network focus on the most important patterns, without being distracted by unnecessary details.
This not only makes the model faster and more efficient but also helps prevent overfitting.
Pooling ensures that C-N-N's work with streamlined, meaningful data, making it easier to learn key features from images.

Pooling in Convolutional Neural Networks, or C-N-N's, comes in two main types: Max Pooling and Average Pooling.
In Max Pooling, we scan a small region of the feature map and select the highest value. This helps the C-N-N focus on the strongest features, making it ideal for tasks like object recognition.
In Average Pooling, we take the average of all the values in the region. This creates a smoother, more generalised feature map, which is helpful when we want a broader understanding of the image.
Both methods simplify the data, but they highlight different aspects depending on the goal.

Let us walk through an example of Max Pooling.
We start with a four-by-four feature map.
Using a two-by-two pooling window, we move across the feature map.
At each step, we select the highest value within that window.
For example, in the top-left region, the highest value is six.
As we shift the window, we keep selecting the highest values, like four and seven.
By the end, we get a smaller, two-by-two matrix showing the strongest features.
This process helps C-N-N's reduce data size while keeping the most important information.

Now, let's look at how Average Pooling works with a feature map.
We begin with a four-by-four grid, with numbers like one, three, two, and four on the top row.
We apply a two-by-two window that slides across the grid.
For each window, we calculate the average of the four numbers inside.
For example, if the window covers one, three, five, and six, the average is three point seven five.
When we move the window, we calculate new averages like two point two five, four point five, and three point seven five.
This gives us a smaller two-by-two output matrix.
Average Pooling smooths out the details, helping the C-N-N's build a simpler, more general understanding of the image.

Pooling is a vital step that makes C-N-N's more effective at handling large and complex images.
After convolution, pooling simplifies the data by reducing the size of feature maps, which makes the model faster and more computationally efficient.
It also helps C-N-N's focus on the most important features, like edges and textures, while ignoring minor details.
By doing this, pooling reduces the risk of overfitting and helps the model generalise better to new data.

Overall, pooling plays a key role in ensuring that C-N-N's can recognise patterns accurately, even when dealing with complex datasets.

Padding in convolutional neural networks means adding extra rows and columns around the edges of the input image. This lets filters scan the borders properly, instead of ignoring them. Without padding, the image size would shrink after each convolution, making the model lose important information.
Padding helps keep the image size stable and ensures that even features near the edges are captured clearly. It is especially important for building deeper, more powerful networks.

Padding can be done in two main ways: Valid Padding and Same Padding.
In Valid Padding, no extra pixels are added, so the feature map becomes smaller after convolution. This is useful when a smaller output is acceptable.
In Same Padding, zeros are added around the edges of the image. This keeps the output size the same as the input size. Same Padding is especially important in deeper networks, where keeping feature map sizes consistent makes the model easier to design and manage.

Let's see how same padding works using a three-by-three filter.
We start with a three-by-three input feature map, with values from one to nine.
By adding a border of zeros around the edges, we expand this to a five-by-five grid.
When we apply the three-by-three filter, the output keeps the original three-by-three size.
This way, the filter can scan the entire border without shrinking the feature map.
Same padding helps the C-N-N preserve spatial details and capture important features at the edges of the image.

Padding plays a key role in controlling the size of feature maps in convolutional neural networks.
Without padding, each convolution step shrinks the output, which can cause important spatial details to be lost in deeper layers.
Padding solves this by preserving the original feature map size across layers, helping the network retain critical patterns, especially at the edges of images.
This consistency makes it easier for the model to learn from fine details and build better representations of complex patterns.

Padding is a crucial step that helps convolutional neural networks, or C-N-Ns, keep important details across layers.
By adding padding around the input, we allow the network to scan the entire image, including its edges, without losing information.
This prevents the feature map from shrinking with each convolution and keeps the spatial structure consistent.
Padding helps C-N-Ns recognise fine details and build better patterns, making them more effective for tasks like image recognition.

Padding might seem simple, but it's a game-changer for building smart neural networks. Every pixel counts — especially at the edges. As you design deeper models, challenge yourself to think not just about layers, but about what you're saving and what you might lose.

Video 4 - Architectures That Shaped CNN Evolution

What makes a C-N-N architecture powerful enough to recognise faces, objects, and even emotions? Let's uncover how clever design choices transform simple data into remarkable predictions — and why every layer plays a vital role.

Alex Net was one of the first deep convolutional neural network architectures to prove the power of deep learning. It showed how large-scale image recognition could be transformed using deep learning techniques. Alex Net's success helped bridge the gap between traditional machine learning and modern deep learning, setting the stage for today's advances in computer vision. This breakthrough sparked a wave of innovation that changed the way machines see and understand images.

Alex Net was developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton at the University of Toronto.
It was introduced in twenty twelve during the Image Net Large Scale Visual Recognition Challenge.
Building on earlier research into neural networks and back propagation, Alex Net demonstrated the power of deep learning for large-scale image classification.
Its success not only broke performance records, but also sparked a major shift towards deep neural networks in computer vision and artificial intelligence research.

Alex Net was trained on the Image Net dataset — one of the largest and most diverse labelled datasets in computer vision.
It included over one point two million images across one thousand categories, ranging from animals and vehicles to household objects.
This was the first time a deep convolutional network was trained on such a massive scale, which demanded high computational power and smart techniques like data augmentation to prevent overfitting.
Alex Net's success proved that large-scale deep learning was not only possible, but highly effective for image recognition.

Alex Net introduced a revolutionary structure with eight layers: five convolutional layers followed by three fully connected layers. Each convolutional layer used ReLU activation to introduce non-linearity and capture complex patterns. Max-pooling layers helped reduce the size of the feature maps, keeping essential information while reducing computation. To handle hardware limits, the network cleverly split across two GPUs for faster training. To prevent overfitting, Alex Net used dropout in the fully connected layers, randomly ignoring neurons during training. These innovations made Alex Net a landmark in deep learning history.

In the 2012 ImageNet competition, Alex Net achieved a top-5 error rate of fifteen point three percent — a major improvement over the second-best model, which had an error rate above twenty-five percent. This huge performance leap proved how powerful deep C-N-Ns could be for large-scale image classification. Alex Net's success didn't just win a competition — it transformed the field. It led to a surge in deep learning research, inspiring new models, smarter techniques, and breakthroughs far beyond image recognition.

V-G-G Net was created by the Visual Geometry Group at the University of Oxford and introduced in twenty fourteen. It became famous for its simple yet powerful approach to C-N-

N design. Instead of using different filter sizes, V-G-G Net relied on small, consistent convolutional filters throughout the network. This consistency helped make the architecture deeper and more effective, setting a new standard in deep learning for image classification.

V-G-G Net uses a deep but simple structure, made up of sixteen to nineteen layers. Each convolutional layer uses a three-by-three filter to capture small details in the image. These filters are stacked, creating a deep network that learns complex patterns. After each block, a max-pooling layer reduces the size of the feature maps, keeping only the most important information. At the end, fully connected layers bring everything together for classification. V-G-G Net's consistent use of small filters helped prove that simple design choices can still achieve powerful results in deep learning.

V-G-G Net was trained on the large-scale Image Net dataset, which contained over one point two million images across one thousand classes. Its deep architecture required significant computational power. However, the results were remarkable. In the two thousand fourteen Image Net competition, V-G-G Net achieved a top five error rate of seven point three percent, outperforming many earlier models. This success showed that deeper networks, when combined with small, consistent filters, could lead to major improvements in accuracy, paving the way for the next generation of deep learning models.

Res Net, short for Residual Network, was introduced by Microsoft Research in twenty fifteen. It changed the way deep learning models were built by solving a major problem — the vanishing gradient. This issue had made it hard to train networks with many layers. Res Net's key innovation was the use of residual connections — shortcuts that skip one or more layers. These connections made it possible to train very deep networks, even those with more than one hundred layers, without losing important information during learning.

Residual connections, also known as skip connections, were the key breakthrough in Res Net. They allow data to skip one or more layers and jump directly to a later point in the network. This helps the model keep important information intact and avoids the problem of vanishing gradients. As a result, Res Net can train much deeper models than before — even with over one hundred layers — without losing performance. These shortcuts made deep learning faster, more accurate, and easier to scale.

Res Net comes in several versions, each suited to different levels of complexity. Res Net fifty is a 50-layer network used for general applications. Res Net one-oh-one goes deeper to handle more complex tasks, and Res Net one-five-two is designed for extremely deep learning with 152 layers. All variants use residual blocks made of two or three convolutional layers. These blocks include shortcut connections that help information flow easily. Res Net also uses batch normalisation, which keeps training stable by adjusting the inputs of each layer. Together, these features make Res Net powerful, scalable, and easy to train—even at great depth.

ResNet introduced several variants tailored for different levels of complexity. **ResNet-50** has 50 layers, making it suitable for many general-purpose applications. **ResNet-101** extends this depth for more complex tasks, while **ResNet-152** is an extremely deep network, capable of capturing intricate features across diverse datasets.

Each variant is built from residual blocks, which consist of two or three convolutional layers with a residual connection. ResNet also uses Batch Normalization, which stabilizes training by normalizing layer inputs, making it easier to train such deep architectures.

Res Net delivered record-breaking performance in the 2015 Image Net challenge, achieving a top-five error rate of only three point six percent. This set a new benchmark for deep learning accuracy. Its architecture showed that very deep networks could still perform well, without losing precision. Res Net's success inspired many future models and proved that residual connections make it possible to build deep networks without sacrificing performance. Its influence continues today, shaping how C-N-N's are designed across many fields—not just in images, but in other complex tasks too.

Now it's your turn — explore how residual connections work by building a simple ResNet block and see how it handles deep learning challenges in action

Video 5 - Solving Deep Learning Challenges with CNN Innovations

Ever wondered how machines can spot a cat, a car, or even your face in a photo? It all starts with the building blocks of C-N-Ns. Let's break down the architecture that powers today's smartest vision systems — one layer at a time.

Let's explore how to build Convolutional Neural Networks using **Keras** and **PyTorch**, two leading deep learning frameworks.
Keras is known for its simplicity and speed — perfect for quick experiments and prototyping.
PyTorch, on the other hand, offers more control and flexibility, especially when working with dynamic computations.
We'll walk through core components like data augmentation, data loaders, and how CNN layers use filters, strides, padding, and pooling. By the end, you'll know how to set up CNNs using tools widely adopted in both industry and research.

Data augmentation increases training variety by applying changes like rotation, flipping, and resizing. This helps reduce overfitting and improves the model's ability to generalise to new data.
In **Keras**, you can use the **Image Data Generator** class or built-in augmentation layers like **t-f dot keras dot layers dot random flip**, **random rotation**, **random zoom**, and **random height**.
In **PyTorch**, data augmentation is handled through **torch vision dot transforms**, using a structure called **transforms dot compose**. This lets you combine multiple changes like **random rotation**, **random horizontal flip**, **random resized crop**, and **to tensor**.
These techniques expose the model to a wider variety of examples, improving learning and performance.

Data loaders help manage large datasets by loading images in batches during training.
In **Keras**, we can use the **Image Data Generator** or the **t-f dot data dot Dataset** application programming interface to load data from directories, resizing and batching it automatically.
In **PyTorch**, the **Data Loader** class works with **Image Folder**, enabling shuffling, parallel processing, and fast, structured loading.
These tools ensure that images are efficiently delivered to the model, improving speed and consistency during training.

Convolutional layers in C-N-N's use filters to extract features such as edges and textures.
The **stride** parameter sets the step size for the filter, controlling how far it moves across the

image.
Padding preserves the size of the feature map by adding extra pixels around the input.
In **Keras**, we use **Conv Two D** to define filters, kernel size, stride, and padding.
In **PyTorch**, we use **torch dot n-n dot Conv Two D** with the same parameters.
These settings help the network learn efficiently while controlling output size and complexity.

Pooling layers in C-N-N's reduce the spatial size of feature maps, keeping only the most important information.
Max pooling selects the highest value in a region, while **average pooling** takes the mean.
These techniques help make the model more efficient and reduce overfitting.
In **Keras**, we use **Max Pooling Two D** or **Average Pooling Two D** for downsampling.
In **PyTorch**, we use **n-n dot Max Pool Two D** or **n-n dot Average Pool Two D** with settings like kernel size and stride to control how pooling is applied.

How do you make a neural network focus on just the important parts of an image? Try experimenting with max and average pooling in Keras or PyTorch, and see how smart downsampling can sharpen model performance.

Video 6: Convolution Neural Networks- Demo

Hello everyone, let us try to understand how a network reads an image and interprets as numerical data, a data which a neural network can easily process.

And we know that convolutional neural networks have been very effective in reading a two-dimension image data.

Let us try to see how we can use the PIL library to open the image file.

To do so, we are going to import some necessary libraries and you know that NumPy and Matplotlib have been very effective in dealing with numbers and visualisation.

Since we are going to deal with arrays, we are going to use NumPy and we are going to visualise our image, we are going to use Matplotlib.

The new library which we are going to introduce today is the PIL library which is required to open the image file.

If you see this command where we are trying to open an image which will convert, which will essentially read the image and it will return you an image object and that image object is something which you can then convert to an array.

This is that operation which goes by to convert the image into a three-dimension matrix.

I have actually uploaded the image here and I am going to execute this.

The moment I execute this, it is going to convert that image into an array.

This is an array of numbers and I am going to simply plot it.

You can see this is an input image. Along with that, you also see the shape of an image array and when you clearly try to look into this shape, you realise that the first dimension here would represent the height, the number of pixels or the number of rows of pixels in this input image.

The second dimension talks about the width that is the number of columns of pixels and the third dimension represents the colour channels that is the depth.

So, you know that since this is a coloured image, it would be a combination of red, green and blue. So, since there are red, green and blue intensity, so you have three as a depth.

So, for each pixel, you have three depth or RGB defined for that.

Now, once we have read the image, you know that in your convolution neural networks, we

use something called as a filter.

A filter is nothing but a convolutional kernel which we apply on the three-dimension image

matrix. So, this is the three-dimension image matrix which I had opened and converted into an array.

This time when I'm trying to open it, I'm trying to open that in a grayscale. A grayscale because that's easy for us to apply filters to, just for the sake of simplicity.

So, while I try to do so, I want to see that how this filter changes the output in terms of dimensions. And this is the code which is going to apply filters on image matrix.

So, after converting that to a grayscale, we are then going to apply a filter.

So, when I convert that to a grayscale, you see the colour part of the image is gone, and you see a grayscale image.

Now, on this image, which is now having a shape of 275 comma 183.

If you remember here, we had the third channel, which is now gone over here because it's a grayscale image.

It's not a coloured image, so no need of RGB values.

I then apply a kernel. This is my kernel, the filter. It's a three cross three filter.

And you have seen that we can use different kind of filters.

This is that filter which does edge detection. Now, what does this filter do?

It will actually slide over this image and compute a weighted sum at each position.

So, it will actually slide like this, keep on sliding, and it will keep on computing

a weighted sum at each position. There are many kinds of filters, but in this example, we are going to use this edge detection filter to highlight where there are edges in the image.

To do so, SciPy provides us something called as convolve2d, which we had imported over

here. So, to perform the convolution operation, we are going to import this, and you're going to see that I am going to apply this convolve2d on my image.

So, this is that kernel which I'm going to convolve over my image.

While I do convolution, you know there is something called as padding.

So, the kind of padding is mentioned over here.

For now, you can skip this. I'll discuss this a little later.

So, while I apply this convolution operation of this kernel on this image, let's try to

see that filtered image.

I'm going to now plot the filtered image. So, now if you see here, this is my filtered image, and all kind of edges you would see, it's actually trying to detect that. So, that has an effect, the change on the image when we are applying convolution.

In real CNNs, we will try to include something called as padding so that our images do not shrink.

So, you see what happened is my original image was 275 comma 183, and you see it turned out to be 273 comma 181.

So, you will see that there is a reduction in the size of an image.

Now, this is due to the convolution operation, which will shrink the dimensions.

How do we preserve the dimension? How do we make sure that whatever is the input dimension is also the same as the output dimension?

That's what we call as padding, and we'll talk about padding shortly.

Let's say this is our original image, and we go by applying this filter.

And now I try to pad my image. That is, I want that this image should not shrink in size.

And that's where I change this mode. You see the code over here, this code,

just copy this code, and you paste it over here.

And just change this to same. From valid, we're going to convert that to same.

Nothing else has changed. When I do that, earlier it was done

without padding, and now this is done with same padding.

Now you see, we have 275 comma 183. So, if you remember this formula, why did

this happen?

If you remember this 273 comma 181, I try to just show you the formula.

You know that the formula goes by something like this.

The output height would be the input height minus the filter height, that is the kernel height, plus 1. Now, if you remember the input height, which is 275 minus the filter height is 3, because it's 3 cross 3 kernel plus 1.

Now if you try to evaluate this, you would see that it would be 273. So, that's how you reach to 273. And the same thing, exact same thing, if you copy that for the width, instead of height, you can have a width. So, this was 183 minus 3 plus 1, that's 181. So, that's what you have, correct?

Now, if I go by applying the same formula, but now this is with padding here.

So, how does it look like? So, when you're going to apply this with padding, that has a different effect, right? So, now because you are also including the padding output.

So, if you see what should happen is that once you're trying to apply padding, so padding is basically going to add those zeros around your image, right?

So, what is going to happen is that we are going to, instead of going to adding 1, we're going to add 3 to it, right?

So, that actually, you see negates the effect of your filter height.

So, when you're trying to subtract that filter height, the padding is going to add to it, subtract the filter height, it's going to add to it.

With that, what happens is your final image dimension remains the same.

So, you have 275, 183. So, it's going to have a padding around this, and that padding is going to make sure that your output remains the same.

That's the reason why padding is very critical in your CNNs.

Now, there are different types of padding. I'm not going to go into that, but imagine it's a zero padding, which is the most common padding where you pad your image with zeros.

And it's very critical in CNNs, especially in the early layers, where we want to maintain

the resolution so that I can extract finer details of image. Now, this is basically the filtered image with that same padding, and the shape is 273, 181.

Now, let's try to talk about different filters. We know that we have discussed the edge

detection filter. In a real convolution neural network, each convolution

layer would apply multiple filters in parallel to the same input.

Here, we are going to take three different filters, each of 3 x 3 size.

This is an edge detection filter. This is a box blur filter, and this is called as a Sobel filter.

So, these numbers, if you are good in your matrix operations, linear algebra, you will realise that these numbers would actually perform the job of detection of edges, blurring the boxes, and detecting horizontal edges, which is your Sobel filter.

So, I have combined these three filters, and for each filter, I am going to perform a two-dimension convolution on the same input image.

And I am going to do this twice. Once with valid padding, that is, the output

would shrink because the filter cannot be where it fully fits. And the next one would be the same padding, where the output keeps the same spatial dimension by adding zero padding around that image. So, I go by each of these kernels one by one, and I apply the valid padding and the same padding. Remember, this is going to shrink the image. This is going to preserve this dimension of the input image. And I'm going to plot that. Now, along with that, I'm also going to plot the shape of these images.

Finally, whatever output I got, since I'm going to append that, you see I'm going to append it, right?

For both the cases, I'm going to combine these outputs, and I call them as feature

maps. So, I'm going to stack all the valid output, and I call that feature map as valid feature map, and the same padding output, and I call that as feature map same. Let's try to see that one by one.

So, this is basically my input image shape 275 cross 183, and I apply the first filter with valid padding, and the same filter with same padding. With valid padding, you see that it's going to reduce the size. With same padding, it's going to remain as it is.

Now, you won't actually see the effect of padding a lot over here. But still, if you try to just observe, you would see that this one looks more rich.

If you carefully see, very carefully, you would see that this looks more rich than this one.

This is a bit, a slightly blurred image. This looks more refined, more sharp. Well, we do not need to go into that. That is already taken care by CNNs.

You would see the accuracy and realise that, yes, because the accuracy is high in case

of same padding. We can justify that saying that, yes, our same padding essentially works.

So, after filtering, each filter will produce you one output channel. This is the first output channel from both the kind of paddings, the valid and the same.

Let's see the filter 2 output. This is the filter 2 output, again from valid and from same padding. You can see the same action being performed over here.

And this is the horizontal edge detection, both for valid and for same. So, each of these filters, for each of these padding techniques, is producing you one output channel.

And I am going to stack these outputs, all the valid.

This is valid 1, valid 2 and valid 3. I am going to stack them. Similarly, same 1, same 2 and same 3, I am going to stack them. And let's see, because I am going to stack them, there are three different filters. My output is, if it's valid, this is the height and the width, and then this is your number of filters which are stacked. Three filters are stacked.

So, this is your height and width and the number of filters which are stacked. And this is how the CNN layers can learn multiple patterns. It can be edges, it can be textures, it can be colours, and all of them can be done at once. And this is the foundation of how CNNs build rich multi-channel representation of images, complex images, layer by layer. And by combining and transforming these feature maps, you can learn deeper, more complex patterns. And that's what makes CNNs so powerful for visual tasks.

So, that was the case about filters. The next thing what we are trying to demonstrate is pooling. Now, if you remember, pooling operation is one of the most widely used operations in convolutional neural networks, especially max pooling.

And the purpose is to reduce the spatial dimension for feature map.

Until now, we were trying to preserve that, but now I want to reduce it. While reducing it, I want to reduce it in a least loss manner. I want to retain the most important information, but still reduce it. Why do I want to reduce it?

To make the network computationally efficient and also add a degree of some invariance, translational invariance. To do so, we are going to add a block of pooling window.

So, I'm going to say that pool size is going to be two, that is the pool window would be a two by two, which will move across the input matrix.

And there is something called a stride, that is how far the window would shift at each step.

A stride of two means the window would jump two pixels at a time in both vertical and horizontal directions.

And for each position, we are going to take the maximum value.

So, if you see here, I'm going to pick the maximum value, np.max in that window.

So, I first get the values in that window and try to find out the maximum value.

So, this is the height and the weight expected of the output.

I initialise that output with zeros and I populate that zeros by capturing the window and then within that window, what's the max value. Now, we apply this pooling on our image and then you try to see that the output of the original image, which is 275, 183, after applying a max pooling of 2 cross 2 and a stride of 2, you would see that it would reduce to 137, 91.

You see how effectively it has reduced and you see the image, you won't be able to realise any loss, at least what you can see with the human eye, it's hard to realise that there has been shifts or distortions in the input image.

So, it has reduced the resolution, preserved strong features, and this is something which you stack between convolution layers so that you can progressively down sample the

representation. And this helps to keep the number of parameters manageable.

There are different kinds of pooling. So, there is something called as an average pooling.

Now, here, it works very similar to max pooling, but instead of taking the max value,

you take the mean value, the average of all values.

Now, people tend to do this if they want to produce a smoother version of the feature map and retain more information rather than just the maximum information about the overall presence of a feature.

I just don't want the strongest activation. In fact, I want the overall presence of a feature.

So, when the goal is to downsample while keeping this more subtle variations in the data,

you go with average pooling. Another important kind of pooling is global average pooling, where instead of doing it over a window, we reduce an entire input image, that is your entire feature map to a single number. I just take the average of all the values.

Imagine if it is a multi-channel feature map and I apply it channel wise, each channel will get reduced to a single scale. In modern convolution neural networks, you would see this operation often being used at the end of convolution blocks.

And the reason is because it drastically reduces the number of parameters.

It is also assumed that it prevents overfitting.

So, we are going to summarise this information of each feature map into a single number,

which represents the overall presence of the learned feature.

It is up to you to apply different kind of pooling and then try to observe how the network is able to capture the patterns in your input.

So, that is all from my side. Thank you.